END TERM REPORT - 5 JAN 2026

WILD-ID PROJECT

Guneesh Gupta
Group 5 - BYOP

Section 1: Abstract

In my BYOP project, I have built a bioacoustic classification system (Named as WILD-ID) capable of identifying wildlife species from the sounds they make. My model is more specialised in identifying the different bird species because I trained it on a dataset containing 10127 species of birds all over the planet. The significant challenge in a model like this are immense diversity of species, environmental background noise, and the varying volume of wildlife species. Coming to know about these challenges led me to search for extensive features like PCEN, etc., from some well-established research papers.

The objectives in my project include developing a deep learning model that trains on thousands of audio files of species and understands the characteristics in sounds to identify them. One of the facts that I found quite interesting is that the model visualises sound, i.e., it makes a 2D picture called a spectrogram out of the audio file and learns the behaviour of different audio patterns in it to identify the animals. The objective was also to train my model in such a way that it can understand variations in sounds and adjust itself to various recording qualities.

My project consists of Model A and Model B. For Model A, which is a general classifying system of basic animals and environmental sounds, I have used a basic environment dataset for it. Based on that, my model A will classify what sound it is and predict its label. Furthermore, if the label corresponds to a bird, the information will trigger Model B, which is a pure specialist in identifying Birds which even a human can't normally differentiate between.

My final deliverable includes justification of the spark differences between PCEN vs MEL SPECTROGRAMS, and the display of learnt parameters of PCEN. It also justifies the learning improvement of the model architecture implemented by me, and I have shown its accuracy and F1 scores along with a t-SNE graph.

Section 2: Methodology

1. **Problem Statement**
   To make a model capable of identifying different wildlife species based on the sounds they make. I wanted to make something similar to Shazam. Like Shazam can identify songs on the basis of the audio, my model should identify animals based on their audio. Making this problem can help in trekking to be aware of your surroundings and judge the potential danger. It can be used to even collect data of different species like birds from a forest, etc., and see which biodiversity exists there. Also, it can be used in household settings.

2. **Data Collection and Preprocessing**

   - **Data Sources**: Firstly, I have used the ESC-50 (Environmental Sound Classification) dataset. It includes categories of Animals (Dog, Rooster), Nature Sounds (Rain, Wind), and Urban noises. It has 2,000 audio samples (5 seconds each), organized into 50 classes, with each audio file 5 seconds long. I downloaded this dataset on my device and ran 50 epochs on it in my Model A.

     Secondly, to make Model B (The bird specialist), I have used the Xeno Canto Bird Sound Dataset. I used the version hosted on Hugging Face by the repository ‘ ilyassmoummad/Xeno-Canto-6s-16khz’. This repository contains thousands of crowd-sourced audio recordings of 10127 birds on our planet. Mostly, the files are 6 seconds long with 16K as the sample rate, equalling 96K samples in each audio file. The data is ingested in a streaming format due to its massive size (hundreds of gigabytes), preventing full-memory loading. I got 32 files at once from the internet webdataset and processed them using ‘torch.load’ techniques to make out the corresponding tensors.

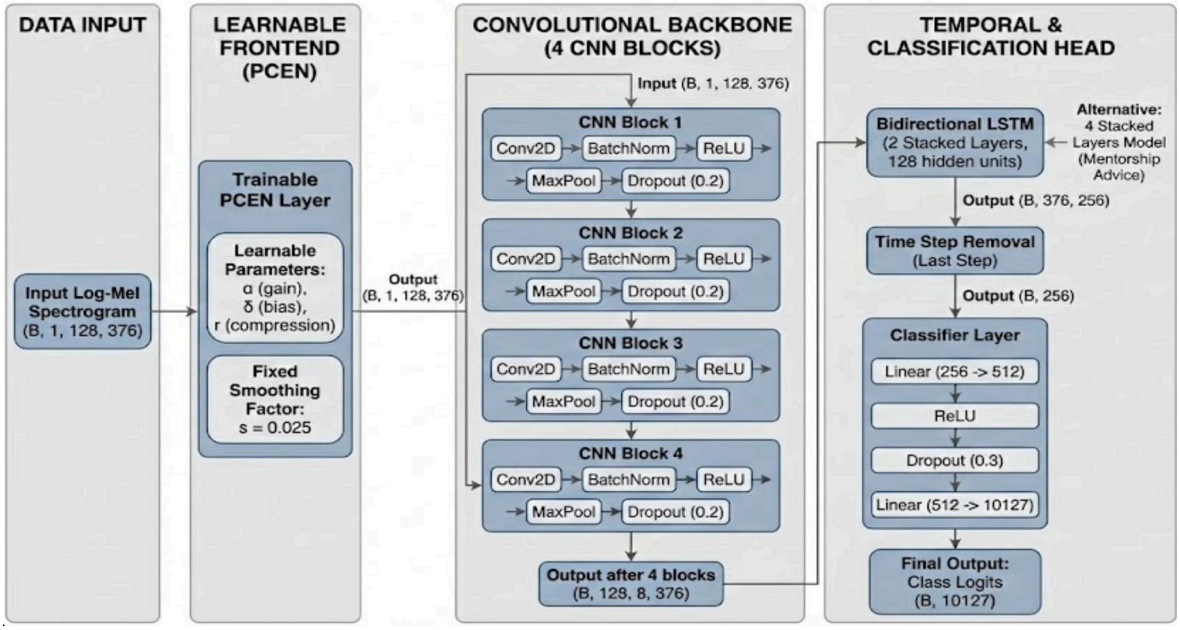
   - **Preprocessing Steps**: Upon loading each batch of size 32, the tensors corresponding to them are made. All audio tensors are processed to replace NaN or infinite values with zero or valid bounds (-1.0 to 1.0). This prevents gradient explosions caused by corrupted crowd-sourced files. The audio files are then cropped or padded to make them exactly 6 seconds long. Then Mel Spectrogram transformation is done with 128 Mel bands, n_fft size of 2048, and a hop length of 256, and they are log-scaled properly. The,n augmentation is done using SpecAugment to mask some parts of the frequency columns or time rows. Mostly, I had kept the time masking parameter to be 90 and the frequency masking parameter to be 35. Now this generates an output tensor of size (32, 1, 128, 376).

3. **Algorithms and Techniques**

The final deliverable includes a Convolutional Recurrent Neural Network (CNN+LSTM). This architecture was selected to include the dual nature of audio spectrograms: they are visual representations (CNN) that unfold over time (LSTM). The model uses a Trainable Per-Channel Energy Normalisation (PCEN) layer as its entry point. It is unlike normal architectures. In my model, the parameters of PCEN can be trained in backprop. As already mentioned, due to the xenocanto dataset's massive size (hundreds of gigabytes), standard in-memory loading was infeasible. So I streamed it from Hugging Face webdataset in batch sizes of 32. All the preprocessing techniques are mentioned in the previous section. For the evaluation phase, I have calculated the top-1 accuracy, top-5 accuracy, and the weighted F1 score. I have also shown the t-SNE graphs for my models.

4. **Model Development**

Architecture Details:



The learnable frontend consists of the trainable PCEN layer. It learns three parameters per frequency band: alpha (gain control), delta (bias), and r

(compression), alongside a fixed smoothing factor s = 0.025. The input and the output are of the same size (B, 1, 128, 376)

The backbone consists of 4 Convolutional Blocks, each Conv2D to BatchNorm to ReLU to MaxPool to Dropout. The input is (B, 1, 128, 376) and the output after 4 blocks of CNN is (B, 128, 8, 376). Each CNN block looks like:-

```
self.block2 = nn.Sequential(
    nn.Conv2d(64, 128, kernel_size=5, padding=2),
    nn.BatchNorm2d(128),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=(2, 1)),
    nn.Dropout(0.2)
)
```

Then, a Bidirectional LSTM with 2 stacked layers and 128 hidden units. I also trained one more model having 4 stacked layers of LSTM as advised by my mentor. It outputs a tensor of size (B, 376, 256) from where we remove the time step to make it (B,256)

```
# 4. Classifier
self.classifier = nn.Sequential
    nn.Linear(128 * 2, 512),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(512, num_classes)
)
```

I use a classifier layer having linear output to 512 from 256, then ReLU, then dropout normalisation, then a linear layer to make the output 10127 from 512.

- **Optimization**:

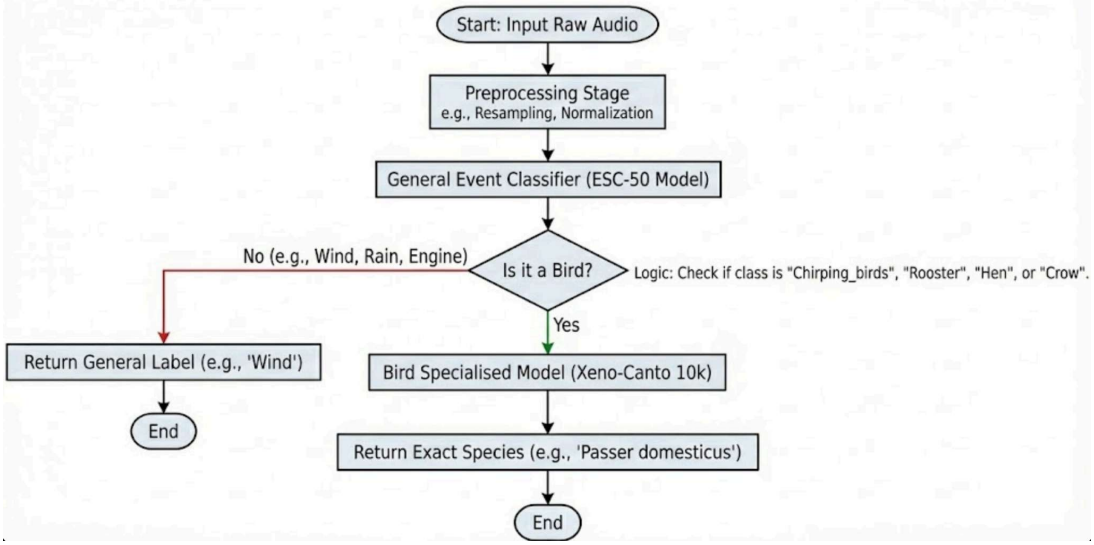
  For finding loss, I have used cross-entropy loss (nn.CrossEntropyLoss). I used the Adam (Adaptive Moment Estimation) optimizer with a converging rate of 5e-5. For Normalization, I have used Batch Normalization to normalize the output to have zero mean and unit variance. I also used Dropout Normalisation with CNN dropout (0.2) to randomly freeze 20 percent of neurons in CNN layers and (0.3) in the classifier layer to freeze 30 percent of neurons there in a trial to reduce overfitting.

- **Evaluation**:

  For evaluation, I have found out the top-1 accuracy, top-5 accuracy, and the weighted F1 scores of my models. The Weighted average calculates the F1-score for each of the 10,127 classes independently. Then it averages them,

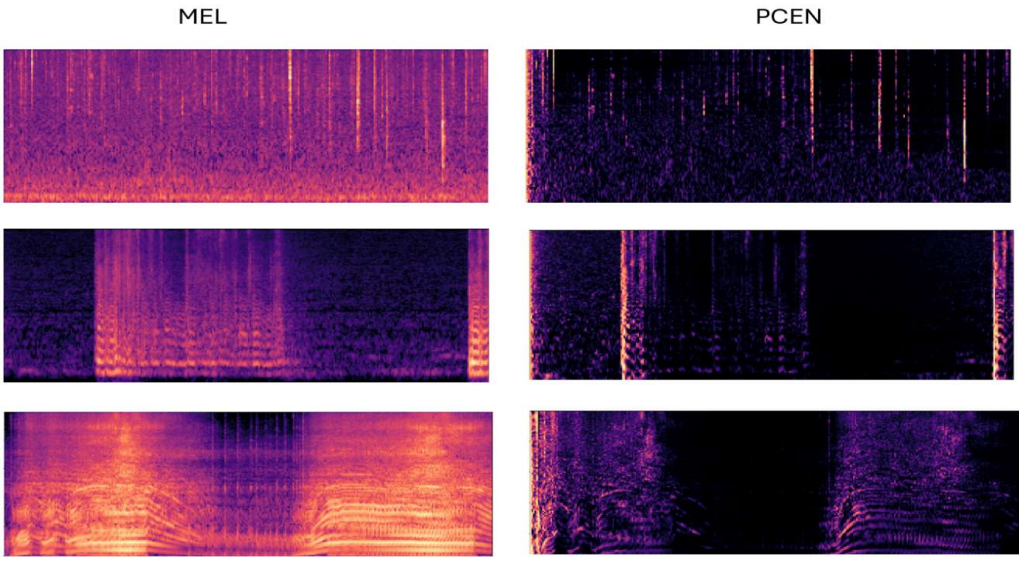
weighting each class by its number of true instances. I have also plotted a t-SNE graph to visualise the latent space analysis.

Implementation Workflow

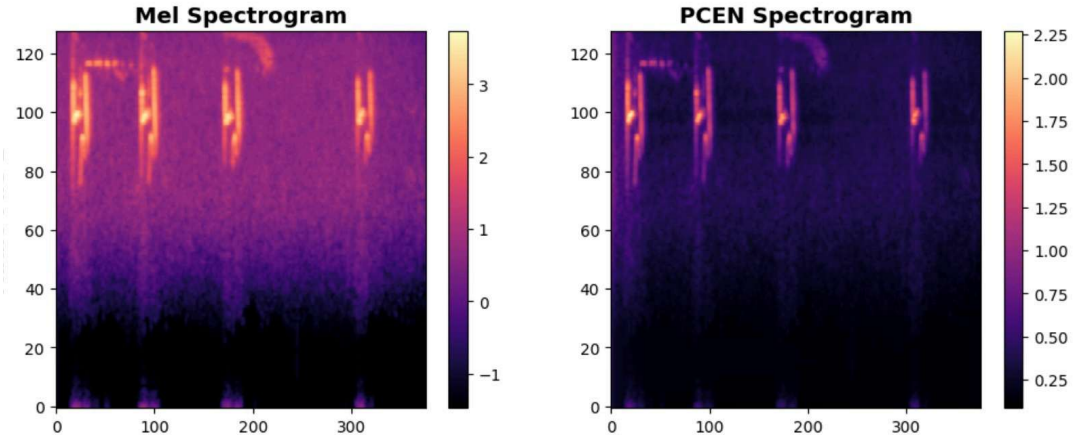


Section 3: Results

I was able to perfectly make spectrograms and point out the difference between MEL and PCEN spectrograms. You can clearly see the difference between the 2 spectrograms made using magma colour pattern in matplotlib. PCEN has been proven to cancel out the external noise in the audio files and make the main audio more prominent. This can easily make the model much better and more robust to different scenarios in the wildlife

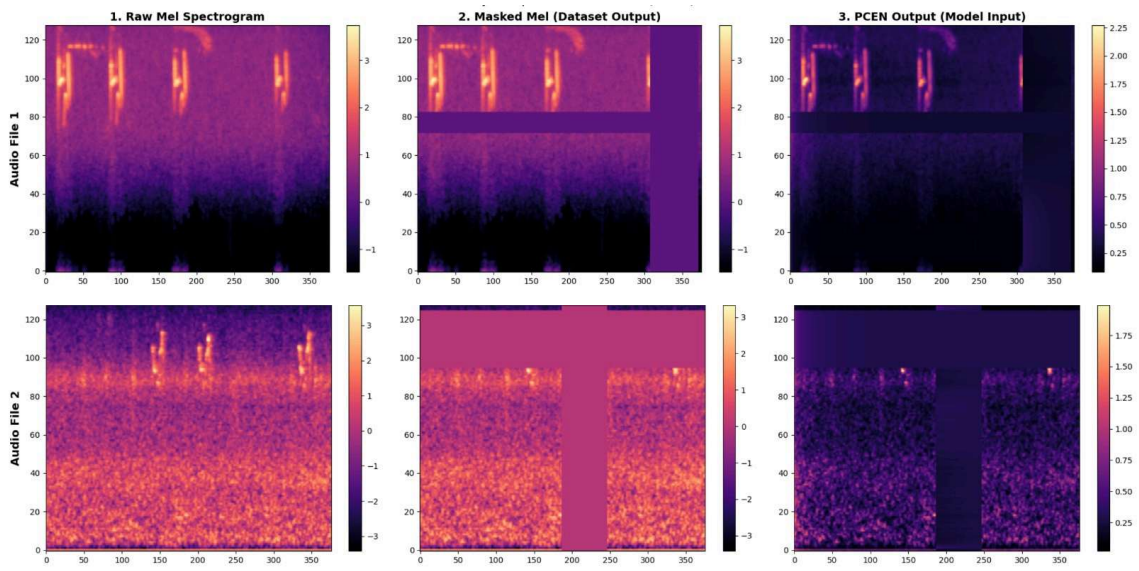


PCEN Demonstration using the built-in function in librosa on files in ESC-50



PCEN Demonstration of an audio file in XenoCanto webdataset by my implementation of its mathematical equation (I did not use any standard library here)

I was also able to generate the masked spectrograms using SpecAugment and show their spectrograms



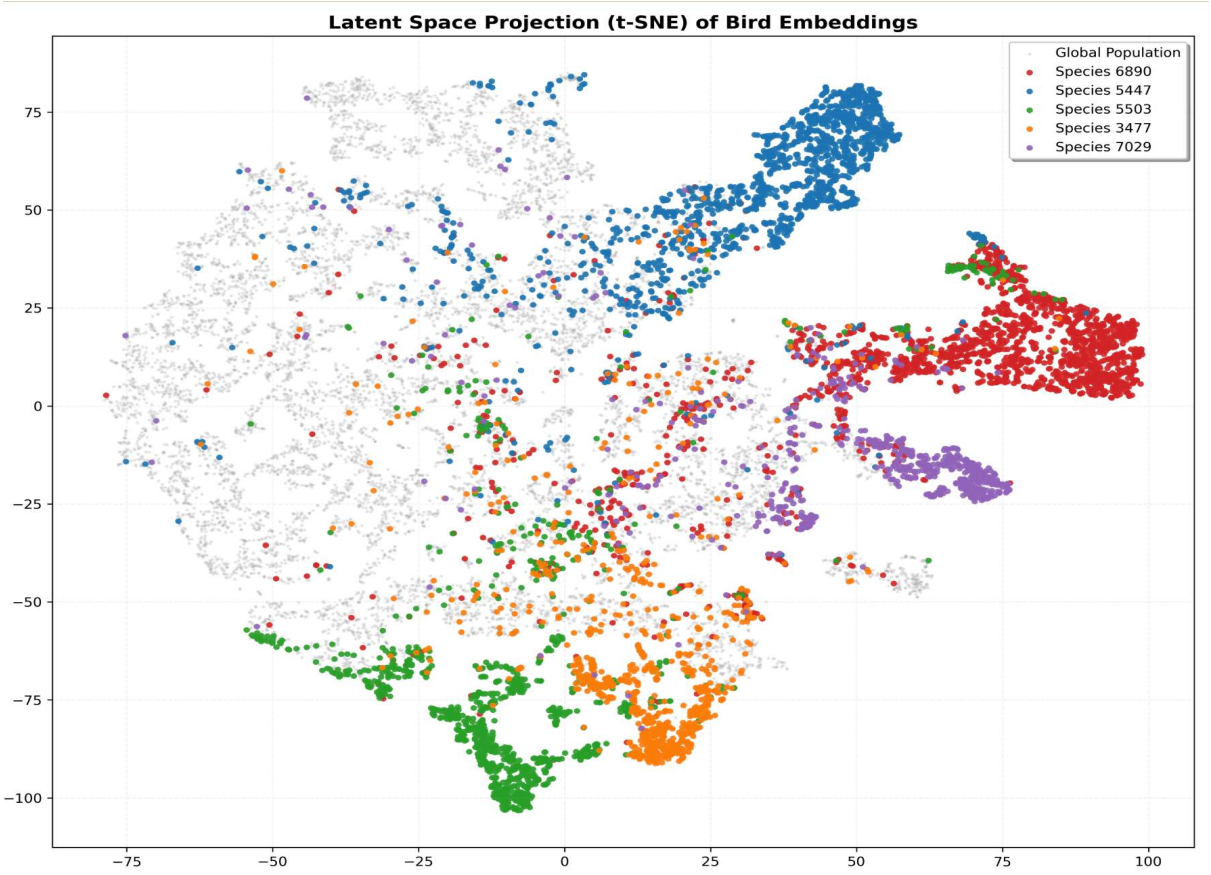
This proves how my data augmentation using SpecAugment was being carried out. I used to make a mel spectrogram, then mask it using SpecAugment, and then make its PCEN output. I also noticed that when I used to enforce a stricter mask (with more masking), the training accuracy went a little down, and the validation set accuracy went a little up in each epoch,

each batch. This proves that using data augmentation, we force the model to learn the exact correct features of our data input and not memorise the noise

APLHA = 0.74318 **DELTA** = 1.03754 **r**= 1.06582
These are the trained parameters of the PCEN layer, showing that they worked the best for the model

Model Configuration	EPOCHS trained	Top-1 Accuracy	Top-5 Accuracy	F1 score (weighted)
2 LSTM layers, 1 classifier layer	85	0.3934 (39.34%)	0.6084 (60.84%)	0.3972
4 LSTM layers, 4 classifier layers	45	0.2094 (20.94%)	0.3604 (36.04%)	0.1934

PERFORMANCE METRICS FOR MY 2 MODELS



t-SNE GRAPH FOR MY MODEL WITH 256 LATENT SPACE DIMENSIONS

Section 4: Final Deliverable

The final deliverable includes all my ESC-50 training codes, my XenoCanto training codes, the .pth files of the best weights of my models, and the results which I got. I have created a GitHub repo on which I will be uploading them.

Section 5: Challenges Faced

One of the major challenges I faced was that the complete Xeno-Canto dataset exceeded the RAM and disk capacities of standard training environments. It was not possible to take the file recording from https://xeno-canto.org, so as a solution, I shifted to streaming from the webdataset of Huggings face library and getting small batches of 32 on my RAM and deleting them after use. In this way, I had to focus specifically on birds because the dataset had majorly bird recording.

Also, I faced a difficulty with the length of the audio recordings in my Xeno-canto dataset. This was not a problem in the ESC-50 dataset, but when I started discovering about Xeno-Canto, I learned that most of recording was of 6 seconds in it but some greatly exceeded this time limit. So as a solution, I had to use padding and cropping to make each file in the processing stage a 6-second audio clip.

During streaming, the pipeline occasionally encountered corrupt audio files and dodging them was a major challenge. It used to crash my code mid-way during training, so it wasted a lot of my time in training, which is basically the reason why I couldn't train more. I used to get the notification of a cancelled run of a notebook midway after 1-2 hours, and it was extremely frustrating. As a result, I wrapped the datalaoder with various try-except blocks and tried various times to debug it on a lot of days.

In a streaming context, ensuring that the Train/Validation/Test splits are proper was difficult. Since we cannot see the whole list of files at once, a random split might assign the same file to "Train" in epoch 1 and "Test" in epoch 2, causing data leakage. So I had to use the zlib.adler32 hash of the file's unique ID and grouped the data to train,val, test sets (e.g., any file hashing to <10 was always the Test set file)

Also, a few times during the training phase, I used to get a problem of Gradient explosion, so all audio tensors are processed to replace NaN or infinite values with zero or valid bounds (-1.0 to 1.0) in my code.

To be frank, I generated the zlib-adler32 hashing code and the debugging of the try-except code snippet with the help of Gemini LLM.

Also, I had spent a great part of my timeline on learning and understanding new things, being a beginner in this field. But this project has made me come even more in love with AI models and deep learning theory.

Section 6: Secondary Goals and Future Scope

I would have loved to train my model a lot longer and with more epochs. I had in mind from the start of the project to generate **confusion matrices** and train my model specifically on that, but due to time limitations (I was stuck a lot on the challenges part), I couldn't implement it. It would have definitely helped the model to perform better on the classes it is confused about right now, and could have generated a better organised t-SNE graph.

Also, I would have liked to train the second model (4 classifier model) to equal the epochs of the initial model and create a proper comparison report between them and show how adding a detailed LSTM and Classifier layer could help in retaining more memory.

Also future scope of this project can be linked to a popular Kaggle contest called BIRDCLEF, in which a similar model is made to listen to the audio and identify remote or understudied birds in places around the earth. But I do understand that a lot of experiments were missing in my work due to time pressure. There would be a better architecture which can increase the accuracy, which I couldn't find yet.

Section 7: References

Understanding Deep Learning :

- **Source**:- **DEEP LEARNING SPECIALISATION** Course by ANDREW NG and **Umich CV lectures EECS 498-007** by Prof. Justin Johnson

The PCEN Layer :

- **Citation:** Wang, Y., et al. (2017). **Trainable Frontend For Robust and Far-Field Keyword Spotting.** *Google Research, ICASSP 2017.*
- **Source:** https://arxiv.org/abs/1607.05666

The Augmentation (SpecAugment):

- **Citation:** Park, D. S., et al. (2019). **SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition.** *Google Brain, Interspeech 2019.*
- **Source:** https://arxiv.org/abs/1904.08779

The Architecture (CRNN):

- **Citation:** Cakir, E., et al. (2017). **Convolutional Recurrent Neural Networks for Polyphonic Sound Event Detection.** *IEEE Transactions on Audio, Speech, and Language Processing.*
- **Source:** https://arxiv.org/abs/1702.06286
